
Module 05: SvelteKit Routing

Master file-based routing, layouts, server-side data loading, and dynamic routes.

Revolution Trading Pros - Web Dev Course

Overview

SvelteKit uses a file-based routing system where the structure of your `src/routes/` directory maps directly to the URLs of your application. This approach is intuitive, scalable, and eliminates the need for a separate router configuration file. In this module, you will learn every routing concept you need to build complex multi-page applications.

File-Based Routing

Every route in your application corresponds to a directory inside `src/routes/`. The URL path is determined by the directory structure:

Project Structure

```
src/routes/
% % % +page.svelte           !' /
% % % about/
%   % % % +page.svelte       !' /about
% % % blog/
%   % % % +page.svelte       !' /blog
%   % % % [slug]/
%       % % % +page.svelte    !' /blog/my-first-post
% % % dashboard/
%   % % % +page.svelte       !' /dashboard
%   % % % settings/
%       % % % +page.svelte    !' /dashboard/settings
%   % % % profile/
%       % % % +page.svelte    !' /dashboard/profile
% % % api/
%   % % % users/
%       % % % +server.ts      !' /api/users (API endpoint)
```

Each directory can contain special files that SvelteKit recognizes. These files follow a naming convention that starts with a plus sign (+).

+page.svelte - Page Components

The `+page.svelte` file defines the UI for a route. It is a regular Svelte component that renders when the user navigates to that URL.

```
<!-- src/routes/about/+page.svelte -->
<script lang="ts">
  // Component logic
  let title = "About Us";
</script>

<svelte:head>
  <title>{title}</title>
  <meta name="description" content="Learn about our company" />
</svelte:head>

<main>
  <h1>{title}</h1>
  <p>Welcome to our about page.</p>
</main>

<style>
  main {
    max-width: 800px;
    margin: 0 auto;
    padding: 2rem;
  }
</style>
```

+layout.svelte - Shared Layouts

Layout files wrap page components and persist across navigations. They are perfect for navigation bars, sidebars, and footers that appear on every page.

Svelte

```
<!-- src/routes/+layout.svelte -->
<script lang="ts">
  import type { Snippet } from "svelte";

  let { children }: { children: Snippet } = $props();
</script>

<header>
  <nav>
    <a href="/">Home</a>
    <a href="/about">About</a>
    <a href="/blog">Blog</a>
    <a href="/dashboard">Dashboard</a>
  </nav>
</header>

<main>
  {@render children()}
</main>

<footer>
  <p>&copy; 2025 My App. All rights reserved.</p>
</footer>

<style>
  header { padding: 1rem 2rem; background: #f5f5f5; }
  nav { display: flex; gap: 1rem; }
  main { min-height: 80vh; padding: 2rem; }
  footer { padding: 1rem 2rem; text-align: center; color: #666; }
</style>
```

Layouts are hierarchical. A layout in `src/routes/dashboard/+layout.svelte` will nest inside the root layout, allowing you to create layout hierarchies for different sections of your app.

Nested Layouts

```
<!-- src/routes/dashboard/+layout.svelte -->
<script lang="ts">
  import type { Snippet } from "svelte";
  let { children }: { children: Snippet } = $props();
</script>

<div class="dashboard">
  <aside>
    <nav>
      <a href="/dashboard">Overview</a>
      <a href="/dashboard/settings">Settings</a>
      <a href="/dashboard/profile">Profile</a>
    </nav>
  </aside>
  <section>
    {@render children()}
  </section>
</div>

<style>
  .dashboard { display: grid; grid-template-columns: 250px 1fr; }
  aside { background: #1a1a2e; color: white; padding: 2rem; }
  section { padding: 2rem; }
</style>
```

+page.server.ts - Server-Side Data Loading

The +page.server.ts file runs exclusively on the server. Its load function fetches data that the page component can access via \$props(). This is SvelteKit's primary data loading mechanism.

TypeScript

```
// src/routes/blog/+page.server.ts
import type { PageServerLoad } from ".$types";

export const load: PageServerLoad = async ({ fetch }) => {
  const response = await fetch("/api/posts");
  const posts = await response.json();

  return {
    posts,
    totalCount: posts.length
  };
};
```

Svelte

```
<!-- src/routes/blog/+page.svelte -->
<script lang="ts">
  // Data from +page.server.ts load function
  let { data } = $props();
</script>

<h1>Blog ({data.totalCount} posts)</h1>

{#each data.posts as post}
  <article>
    <h2><a href="/blog/{post.slug}">{post.title}</a></h2>
    <p>{post.excerpt}</p>
  </article>
{/each}
```

The load function has access to useful parameters including fetch (a special fetch that works server-side), params (route parameters), cookies, url, and the request object.

+layout.server.ts - Layout Data Loading

Just like pages, layouts can have server-side load functions. Layout data is available to all child pages and nested layouts.

TypeScript

```
// src/routes/+layout.server.ts
import type { LayoutServerLoad } from ".$types";

export const load: LayoutServerLoad = async ({ cookies }) => {
  const sessionId = cookies.get("session_id");

  if (sessionId) {
    const user = await getUser(sessionId); // Your auth logic
    return { user };
  }

  return { user: null };
};
```

Svelte

```
<!-- Any page can access layout data -->
<script lang="ts">
  let { data } = $props();
  // data.user is available from the layout load function
</script>

{#if data.user}
  <p>Welcome, {data.user.name}!</p>
{:else}
  <a href="/login">Log In</a>
{/if}
```

Dynamic Routes with [slug]

Dynamic route segments are defined by wrapping a directory name in square brackets. The value is captured as a parameter and available in load functions.

Project Structure

```
src/routes/
% % %  blog/
%      % % %  +page.svelte          !' /blog (list all posts)
%      % % %  [slug]/
%          % % %  +page.svelte      !' /blog/my-post (single post)
%          % % %  +page.server.ts   !' Server data loading
% % %  users/
%      % % %  [id]/
%          % % %  +page.svelte      !' /users/123
```

TypeScript

```
// src/routes/blog/[slug]/+page.server.ts
import type { PageServerLoad } from "./$types";
import { error } from "@sveltejs/kit";

export const load: PageServerLoad = async ({ params }) => {
  const { slug } = params; // e.g., "my-first-post"

  const post = await getPostBySlug(slug); // Your data fetching

  if (!post) {
    error(404, { message: "Post not found" });
  }

  return { post };
};
```

Rest Parameters

Catch all remaining path segments with [...rest]:

```
// src/routes/docs/[...path]/+page.svelte
// Matches: /docs/getting-started
//           /docs/api/reference/v2
//           /docs/anything/nested/deeply
```


Route Groups

Route groups let you organize routes without affecting the URL structure. A group is created by wrapping a directory name in parentheses.

Project Structure

```
src/routes/
% % % (marketing)/          # Group - not in URL
%   % % % +layout.svelte    # Marketing layout
%   % % % +page.svelte      !' /
%   % % % about/
%   %   % % % +page.svelte   !' /about
%   % % % pricing/
%       % % % +page.svelte   !' /pricing
% % % (app)/                # Group - not in URL
%   % % % +layout.svelte     # App layout (with sidebar)
%   % % % dashboard/
%   %   % % % +page.svelte   !' /dashboard
%   % % % settings/
%       % % % +page.svelte   !' /settings
```

This is powerful because the marketing pages and app pages can have completely different layouts without sharing URL prefixes. The (marketing) group uses a simple header/footer layout, while the (app) group uses a sidebar layout.

Error Pages

SvelteKit provides built-in error handling through +error.svelte files. These display when an error occurs during loading or rendering.

```
<!-- src/routes/+error.svelte -->
<script lang="ts">
  import { page } from "$app/stores";
</script>

<div class="error">
  <h1>{$page.status}</h1>
  <p>{$page.error?.message}</p>
  <a href="/">Go Home</a>
</div>

<style>
  .error {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    min-height: 60vh;
    text-align: center;
  }
  h1 { font-size: 4rem; color: #ff3e00; }
</style>
```

Error pages follow the layout hierarchy. You can have specific error pages at different levels: a root `+error.svelte` for general errors, and nested `+error.svelte` files for section-specific error handling.

Throwing Errors in Load Functions

TypeScript

```
import { error, redirect } from "@sveltejs/kit";

export const load = async ({ params, locals }) => {
  // 404 error
  if (!found) {
    error(404, { message: "Page not found" });
  }

  // 403 error
  if (!locals.user) {
    error(403, { message: "You must be logged in" });
  }

  // Redirect instead of error
  if (shouldRedirect) {
    redirect(307, "/login");
  }
};
```

Module 5 Exercise

Exercise: Multi-Page Application

Put what you have learned into practice with this hands-on exercise.

- Step 1: Create a new SvelteKit project with a root layout containing a navigation bar.
- Step 2: Create these routes: / (home), /about, /blog, /blog/[slug], /contact.
- Step 3: Create a (marketing) route group for /, /about, and /contact with a simple layout.
- Step 4: Create a `+page.server.ts` for /blog that returns a list of mock blog posts.
- Step 5: Create a dynamic `/blog/[slug]` route that loads a single post.
- Step 6: Add a 404 error page with a custom `+error.svelte`.
- Step 7: Add `svelte:head` with title and meta tags to each page.
- Step 8: Test navigation between all pages and verify data loading works.
- Step 9: Commit your changes to Git.

Summary

- SvelteKit uses file-based routing where directory structure maps to URLs.
- `+page.svelte` defines the UI for each route.
- `+layout.svelte` creates persistent, hierarchical layouts.
- `+page.server.ts` loads data on the server, available to pages via `$props()`.
- Dynamic routes `[slug]` capture URL segments as parameters.
- Route groups (name) organize files without affecting URLs.
- `+error.svelte` provides custom error pages at any level.